

Group Name: Player

Project 1 Part 2 Documentation

Name of the player program: Player (Player.java)

Additional files/program: State.java, GameConstants.java

I. Members of the Group

Quan Dinh

- Implemented the base data-structure for the Lasker Morris game: State's attributes, mill and neighbor dictionaries.
- Wrote the functions for getting successor of game states, and the utility evaluation function
- Implemented heuristic search strategy (iterative deepening) based on time limits
- Researched heuristic evaluation weights and factors for the game.

Elijah Gray

- Assisted with the writing of functions such as the function that helps handle opponent moves.
- Did bug testing and made suggestions on findings.
- Did experiments with an experimental branch.
- Made code that was integrated into the final branch.
- Helped organize the group and facilitated initial communication.
- Did research on heuristics and strategies.
- Helped improve the heuristics.

Joseph Pisano

- Wrote the functions for checking the validity of moves from both the opponent and the ones generated by the player to make sure they are legal moves.
- General Bug Fixing
- Wrote documentation and added additional comments throughout the code.

II. Instructions for Compilation

Use any java compiler with the Player.java file:

1. Run the file Player.java in the compiler (i.e IntelliJ)
2. The program will start and the user can interact with the program through the terminal. The user can type "orange" or "blue" to tell the program which stone it should be.
3. The user can interact with the program to play Lasker Morris (by entering the specified move format)

To run the program with referee:

1. Enter “**cs4341-referee laskermorris -p1 "java Player.java" -p2 "java Player.java" --visual**” in terminal to run the referee.
2. The referee will call two of the Player program to play against each other.
3. Once the game ends, the referee will return the result of the game.

III. Utility and Evaluation Function

The code uses the function `checkUtility()` which calls a series of other functions to calculate the evaluation criterias for a given state of the game. The heuristic evaluation function also acts as a utility function for terminal states, as it also considers the utility for game-ending states (through multipliers).

These evaluation criterias are:

1. Closed Mill: Utility if the mill was last closed by a player, and a stone is captured
2. Number of Mills: The difference in the number of mills
3. Number of Blocked Pieces (Immobilized): The difference in the number of blocked pieces
4. Number of Pieces Left: The difference in the number of pieces left
5. Two & Three-piece Configuration: The difference in configuration that only needs 1 stone to form a mill (If 2 two-piece configurations share a common stone, then it is also a three-piece configuration).
6. Number of Double Mills: The difference in the number of mills that share a common stone
7. Winning the Game: Utility if the state concludes with a player winning. This is the most dominant heuristic as its return value is multiplied by a value greater than all other heuristics combined.

Depending on the stones in hand, these criterias are multiplied by various values in `checkUtility()` in relation to how beneficial that state is to the player. The evaluation is divided into 3 major stage:

1. Stone(s) Left in Hand: This heuristic focuses on taking control: making good setups for two & three-pieces configuration, capturing stones, and blocking opponent's mills. There is less weight to getting blocked. To achieve that goal, the multipliers are:
 - Closed Mill x 40
 - Number of Mills x 20
 - Number of Blocked Pieces x 3
 - Number of Pieces Left x 9
 - Two-piece Configuration x 10
 - Three-piece Configuration x 7
 - Number of Double Mills x 0

- Winning Game x 1200
2. No Stones Left in Hand: This heuristic focuses on stabilizing: making good defensive moves, capturing stones, and avoiding getting blocked (and blocking). There are less weights for setting up configurations. To achieve that goal, the multipliers are:
 - Closed Mill x 40
 - Number of Mills x 30
 - Number of Blocked Pieces x 10
 - Number of Pieces Left x 11
 - Two-piece Configuration x 0
 - Three-piece Configuration x 0
 - Number of Double Mills x 6
 - Winning Game x 1200
 3. Flying: This heuristic focuses on closing the game: making aggressive moves, capturing stones, and avoiding getting captured. There are less weights for setting up configurations and getting blocked. To achieve that goal, the multipliers are:
 - Closed Mill x 16
 - Number of Mills x 0
 - Number of Blocked Pieces x 0
 - Number of Pieces Left x 0
 - Two-piece Configuration x 10 (to avoid opponent's capturing)
 - Three-piece Configuration x 1 (to avoid opponent's capturing)
 - Number of Double Mills x 0
 - Winning Game x 1200

Note: These heuristics do not mean the player needs to play all their stones before moving stones. However, the stones in the hand allows the player to have a higher advantage that the stones on the board cannot perform, which requires a separate evaluation.

The heuristic utility is the sum of all the heuristic criterias with their multipliers within the stage that the state belongs to.

The heuristic evaluations criterias and multipliers are inspired by this research paper: <http://www.dasconference.ro/papers/2008/B7.pdf>. We modified the factors to fit with a Lasker Morris Game instead of Nine Men's Morris.

IV. Heuristic Strategies to Optimize Time

We use Alpha-Beta pruning and Iterative Deepening to decide on how to expand the tree before the time limit.

- Alpha-Beta pruning: Decrease the number of nodes being evaluated by pruning out “bad” moves early.
- Iterative Deepening: Check if time limit has been exceeded, and use the Minimax Algorithm (Depth-1st Search) to get for the best move in each depth. It will ensure that the program always has a move to be made within the time limit, and incrementally get better moves (if there are any) as the tree expands. If the time limit is exceeded, the algorithm will stop searching and return the best moves from the latest fully explored search.

V. Results

TESTINGS

The program was tested against human players, itself through the referee, and itself through the compiler’s terminal.

1. Playing against humans.
 - We did various different moves, while: playing seriously or losing on purpose to see if the program tries to win (i.e leaving mills purposefully open).
 - The program usually makes optimized moves, which beats the human player.
2. Playing against itself through the referee.
 - The program usually draws against itself.
 - We want to ensure that the program draws against itself since we would like a more balanced playstyle (rather than dominating itself).
 - Testing through the referee gives insights to how we can adjust the heuristic multipliers.
3. Iterating its own moves against the program in the compiler’s terminal (to test heuristics and data-structure)
 - We used the sequence that they always played to test it for ourselves in our custom board in the terminal, and tried out different moves in different intervals to see how the program acts.
 - We printed out the heuristics utility at each depth iteration, and the “best” moves of different depths to check the program’s decision-making. This ensure the Iterative Deepening is working as intended and the heuristic utility makes sense
 - We printed out the main board to see if the program is keeping track of stones and mills correctly. Additionally, with every move, we print the number of stones in hand and on board of each respective player, and the mills that were formed on the current board. This ensures the data structure for the program is being updated correctly (i.e the game itself is working).

- Testing its own move against itself in the compiler ensures that the program is consistent in making moves and whether they are making the most “optimized” moves (which we cannot see on the referee’s visual).

STRENGTH AND WEAKNESSES

1. Strength:

- The program has a very in-depth heuristic evaluation function (and criterias), which allow very strategic plays.
- The program also uses iterative deepening to ensure that there will always be a move to be made, and scales very well with a time-limit.
- The program also has strong data-structures (such as State) that allow ease of access and reduce time-complexity by storing data (i.e mills, moves).
- The program uses dictionaries which allow quick look-ups, and reduce the amount of time traversing loops.

2. Weakness:

- The program has some unoptimized loops that could be avoided or improved by integrating function better (but could lead to more complexity).
- The program has some redundant computation where the same data is re-calculated in different parts. A fix would reduce time complexity.
- The program does not make a different move if both players have not captured a stone in 20 consecutive moves.

VI. Discussion on Chosen Functions:

We did research on different factors that people consider when playing the game, and modified the evaluation function to fit our program/interests (i.e from Nine Men’s Morris to Lasker Morris).

The evaluation function was decided on by testing which heuristics were most vital in leading to a winning state. We tested different heuristic multipliers and tested them against each other and itself to see how the utility weights affect the program’s decision-making.

The various different evaluation criterias allow for more in-depth and strategic thinking. Since we cannot arrive at terminal nodes in every turn, the many different criterias allow the program to make a lot of considerations and closely “estimate” the utility value of each state. (As opposed to a simpler and fewer criterias, which may cause the estimation to deviate due to lack of consideration for important game factors)

We believe that all of our research, testing, and considerations allowed us to pick good heuristics and evaluation functions.

VII. Important notes:

- Due to slow initialization of the program to the referee in the beginning, we set the time limit of the program to ~2.3 seconds, and increased the time limit to 4.9 seconds after the first move was returned. The program is able to handle the full 5 seconds outside this context.
- The function to check legality of moves: `checkLegalMoves()` is fully functional however due to bugs with the referee's initialization as mentioned above does not work with it. So for the code running with the referee the ability to `checkLegalMoves()` has been commented out.